Evaluating RL Explainability Methods by How Much They Help Fix Bugs in Agents

Ram Rachum[1,2], **Yotam Amitai**[3], **Bálint Gyevnár**[4], **Reuth Mirsky**[2] and **Cameron Allen**[1]
[1]University of California, Berkeley
[2]Tufts University
[3]Independent Researcher
[4]Carnegie Mellon University
ram.rachum@berkeley.edu, yotamitai@gmail.com, bgyevnar@cmu.edu, reuth.mirsky@tufts.edu, camallen@berkeley.edu

Abstract

This preliminary paper outlines a planned evaluation benchmark for Explainable Reinforcement Learning (XRL) methods. Current evaluations rely on functionally-grounded metrics like faithfulness and compactness, and on human-grounded proxies like subjective ratings or prediction accuracy. We suggest evaluating XRL methods by how effectively their generated explanations help to diagnose and fix malfunctioning reinforcement learning (RL) agents. We propose **EvalXRL**, a benchmark in which a Large Language Model (LLM) coding agent uses different XRL methods to diagnose a held-out malfunction in an RL agent, and then repair it. Our proposed benchmark iterates across (environment $\times$ malfunction $\times$ XRL method) tuples and uses the reward signal of the RL agents to form a final score for each XRL method. The coding agent may use the method *interactively*: invoke the XRL method, process its output, form new hypotheses on what is broken, and invoke the method again with parameters adjusted for testing these hypotheses. This closed-loop structure may be described as a simplified version of the scientific method. Some XRL methods provide self-evaluations that follow this pattern; we propose the first head-to-head comparison of multiple XRL methods in closed-loop usage.

1 Introduction

XRL methods produce explanations that answer several distinct needs. Developers want to know why their agent is failing; end-users want to know whether an agent is trustworthy; regulators want accountability in systems that affect the public. All these stakeholders want an explanation, but each wants to get something different out of it, so each has different criteria for what they will consider a good explanation [Miller, 2019].

The landscape of XRL methods is as varied as the makeup of its target audience. Methods differ in what kind of questions they answer, in what information they use to build answers, and in the form their answers take. A saliency map [Greydanus *et al.*, 2018] marks which pixels of an observation drove a single action; HIGHLIGHTS [Amir and Amir, 2018] returns short video clips of the policy's most important moments; reward decomposition [Juozapaitis *et al.*, 2019] reports per-component contributions to a scalar value; and VIPER [Bastani *et al.*, 2018] distills the whole policy into a readable decision tree. Given this Cambrian explosion of outputs, the XRL community would benefit from evaluation methods that take in each of these heterogeneous artifacts and produce a homogeneous array of comparable scores. Hopefully, these scores would answer the questions "how useful are the explanations produced by each method?" and "which methods are the most useful?" keeping in mind that the different types of XRL users we described above might each require a different evaluation scheme.

Evaluating explanations is a difficult problem, because an explanation is considered good to the extent that it helps the user *understand*; therefore, each XRL evaluation paradigm can be interpreted as an answer to the epistemological question "what does it mean to *understand*?" [Lipton, 2009; de Regt, 2017] Table 1 surveys three existing paradigms for XRL evaluation, each framed as an implicit answer to this question.

Table 1: Existing paradigms for XRL evaluation, framed as implicit answers to *"what does it mean to understand?"*

Paradigm	**Implicit answer**
Subjective satisfaction and trust ratings [Hoffman *et al.*, 2018]	"If enough people feel that they understand, then they do."
Fidelity and faithfulness measures [Xiong *et al.*, 2024]	"To understand is to create a simpler model that produces similar results as a complex model."
Simulatability tests [Hase and Bansal, 2020; Anderson *et al.*, 2019]	"To understand is to be able to predict the outputs."

We are proposing an evaluation paradigm whose respective answer is: **"to understand a mechanism is to be able to fix it when it breaks."** This functional view of explanation is long-standing in cognitive science [Lombrozo, 2006],

with a recent ability-based formulation for AI [Chen *et al.*, 2026]; as an evaluation paradigm it is partial like the others, but measurable. Therefore, our main focus is the first motivation we listed: the developer's need to know why their agents are failing. This is often called *debugging* [Gyevnar and Towers, 2025], and we will use the terms debugging and *diagnosis* synonymously. Diagnosing failures is an arduous but necessary phase in the endeavor of developing increasingly capable RL agents. In this paper, we propose a method to evaluate which XRL methods produce explanations that are most useful for diagnosis. The framing follows the spirit of Doshi-Velez and Kim (2017)'s *application-grounded* evaluation, with the developer's downstream task as the metric. It echoes recent calls for downstream-validated, fair-fight interpretability evaluation [Marks, 2025; Nanda *et al.*, 2025] and for treating intervention outcomes as ground truth [Barez and others, 2026]. To our knowledge, though, this has not been executed as a benchmark, and not for explainable RL.

This approach offers several benefits. First, we can wire the RL agent's reward signal directly to the output of our evaluation. In other words, we define the success of an XRL method as the performance of an RL agent that was repaired using information from that method. This unmediated connection may be a good strategy for removing proxy-driven bias from our scores. Most excitingly, diagnosis and repair are interactive processes, in which an explainability method may be used multiple times; we elaborate in design choice 2 below.

We propose **EvalXRL**, a benchmark for evaluating XRL methods by how well they help a developer repair online-RL agents. It operationalizes task completion as follows: (1) give the developer a deliberately broken RL agent and an XRL method; (2) ask them to repair it; (3) measure how well the repaired agent performs, as reported by its reward signal or a held-out scoring function. The score is always continuous, never a binary "did you find the bug?" judgment; we never ask the developer to articulate the cause, only to make the agent perform better. Three design choices follow:

1. **Deliberately broken RL agents as evaluation substrate.** We construct agents with known, controlled malfunctions whose diagnosis requires reasoning about behavioral consequences. Holding the malfunction set fixed across all XRL methods lets us measure each method's diagnostic utility on like-for-like cells.

2. **The developer may use the method multiple times, and adjust its parameters.** Diagnosis is a closed-loop process: the developer invokes the XRL method, processes its output, forms new hypotheses about what is broken, and re-invokes the method with parameters adjusted to test these specific hypotheses. This is in contrast to one-shot or pre-planned multi-shot use (open-loop), where parameters are fixed without reference to intermediate results. We make closed-loop use a first-class property of the evaluation: the repair score is computed after arbitrarily many XRL method invocations within the per-session budget, not after one. Some XRL methods provide self-evaluations in a closed-loop pattern [Amitai *et al.*, 2024a; Wu *et al.*, 2022; Dodge *et al.*, 2021; Arzate Cruz and Igarashi, 2021]; we propose the first head-to-head comparison of multiple XRL methods in closed-loop usage.

3. **We use an LLM coding agent as the developer.** Running our proposed procedure with LLM agents is orders of magnitude cheaper and faster than running it with human subjects, and avoids complex approvals for human research. In our default configuration the developer is a frontier LLM (e.g., Claude Opus 4.8) wrapped in a minimalist coding-agent scaffold (e.g., *mini-swe-agent* [SWE-agent Team, 2025]), running in a sandboxed Docker environment with full programmatic control: it can read source, run experiments, modify parameters, retrain the agent, and submit a fix. We measure variance across N runs per cell. We discuss the trade-offs of more elaborate scaffolds in Section 5.

A measurement of how effective an XRL method is for helping an LLM agent fix a bug may not carry over accurately enough to the use case of helping a human, and we sketch a calibration between the two in Section 5. However, we argue that besides serving as a proxy for humans, this measurement may be useful in and of itself. Bhatt *et al.* (2020) surveyed deployed XAI systems and found that the dominant user is the ML engineer who is debugging a model, not a layperson; and LLM coding agents are increasingly doing engineer-shaped work, with clear evidence that engineering of ML systems is being automated with LLM assistance [Kulibaba *et al.*, 2025; Liu *et al.*, 2025], often referred to as "vibe coding". This suggests that LLM coding agents may take a larger role in debugging work that was previously done by human engineers. If "vibe debugging" becomes prevalent, we advocate that it should be studied in controlled settings.

We present EvalXRL as a benchmark *design* and a set of hypotheses, not yet as a validated leaderboard. The pilot run that would test H1–H3 is the immediate next step, and is the contribution of a follow-up paper. The rest of this paper describes the design, the hypotheses, and open questions we want the community's input on.

2 Related Work

The closest prior work to EvalXRL is Gyevnar and Towers (2025), who divide XRL evaluation into two contexts: *debugging* (developers verifying or investigating an agent before/after deployment) and *teaming* (humans and agents collaborating during deployment). Their five debugging metrics are all prediction-based: next-action, goal, sub-goal, counterfactual policy, and time taken. Task completion appears in their layout only on the teaming side. The cell their layout does not contain is *task completion as a debugging metric*: not predicting what the agent will do, but fixing what the broken agent fails to do and then measuring the fixed agent. EvalXRL is an attempt to fill that cell.

Cross-method XRL comparisons. Recent work has compared multiple XRL methods on the same task with the same users via objective accuracy. Septon *et al.* (2023) crossed HIGHLIGHTS with reward decomposition on Highway-env and Pacman; Amitai *et al.* (2024b) found counterfactual-outcome explanations and reward decomposition comple-

mentary on a highway agent; Towers *et al.* (2025) evaluated four mechanisms on Pacman; and Frost *et al.* (2021) compared counterfactual versus critical-state trajectories under distribution shift. EvalXRL extends this comparison style from prediction and identification tasks to repair tasks.

Repair as the XAI use case. Bhatt *et al.* (2020) found that ML engineers debugging models are XAI's dominant real-world consumers. In supervised XAI, Adebayo *et al.* (2020) introduced *debugging tests* that evaluate explanation methods against known model bugs; we adapt this framing to RL by scoring XRL methods on downstream repair success. The closest precursor to EvalXRL is Sequeira and Gervasio (2020), who constructed Frogger agents with controlled deficits and ran a user study where participants identified each agent's capabilities and limitations from XRL-generated visual summaries. Olson *et al.* (2021) extended this to flaw-detection: non-experts who watched counterfactual states from a Space Invaders deep RL agent whose ship-position pixels had been masked raised flaw identification from 57% to 90%. Earlier, Hayes and Shah (2017) generated natural-language policy explanations to help operators pinpoint controller faults, and Arzate Cruz and Igarashi (2021) had non-experts iteratively patch a Super Mario Bros agent with interactive explanations. Amitai *et al.* (2024a)'s ASQ-IT lets users iteratively query an RL agent's behavior, with a user study showing that interactive querying improves users' ability to identify faulty behavior; this directly motivates EvalXRL's closed-loop design choice (Section 1). None of these studies compare XRL methods head-to-head on a controlled malfunction set. Paleja *et al.* (2021) ran the closest task-completion study, but with a hand-designed agent rather than a learned RL policy. Tappler *et al.* (2025)'s LEGIBLE demonstrates that the diagnose-and-repair loop is already viable inside RL, lifting cumulative reward by up to 273% on highway-fast without retraining, though as a single method it offers no head-to-head comparison.

Dialogic and hypothesis-driven XAI. The argument that explanation is fundamentally dialogical rather than one-shot has been made repeatedly. Miller (2019) reviews the cognitive-science evidence that explanation is an abductive-reasoning loop and a social process. Madumal *et al.* (2018) derive a grounded dialog model from 398 explanation dialogs in which follow-up questions and re-explanation are first-class moves. Lakkaraju *et al.* (2022)'s interviews with 26 domain experts (medical and policy researchers) report that all but one cited the impossibility of conversing with explanations as a primary frustration. Miller (2023) argues for a paradigm shift to hypothesis-driven decision support, what he calls *evaluative AI*; Le *et al.* (2024) provide a Weight-of-Evidence implementation reporting improved trust calibration over a recommendation-driven baseline, with mixed accuracy effects across populations. Wu *et al.* (2022)'s LangXRL operationalizes a related idea in RL: users edit history information and observe counterfactual changes in agent behavior, with larger gains on abnormality identification and downstream actionability against a static-text baseline. Dodge *et al.* (2021) extend the structured-iteration argument with AAR/AI, an after-action-review protocol that frames RL agent assessment as Popperian falsification. EvalXRL sits in this lineage but inverts the human/machine roles: the LLM coder, not the human, drives the closed loop, which lets the comparison run at scale and be scored end-to-end on a downstream repair task.

LLM coding agents with tools. A parallel thread asks whether giving an LLM coding agent a debugging or ML-engineering tool improves repair performance. Yuan *et al.* (2025)'s Debug-Gym lifts SWE-bench Lite scores by exposing `pdb`, and Levin *et al.* (2025) let an LLM autonomously drive `pdb`/`gdb`/`lldb` to fix student Python bugs. Haque *et al.* (2025) provide a counterpoint: naive injection of full execution traces helps in only 2 of 6 dataset-model configurations. Chan *et al.* (2024)'s MLE-bench and Wijk *et al.* (2024)'s RE-Bench drop LLM agents into sandboxed ML engineering tasks scored by execution; EvalXRL inherits their harness pattern but re-targets it at XRL-method-assisted repair of malfunctioning RL agents. Hariharan *et al.* (2025)'s Breakpoint takes the closest methodological step: it auto-generates code-repair tasks by corrupting functions in real repositories, using fault-injection rather than naturally-occurring bugs. EvalXRL's malfunction-injection design follows the same logic, applied to RL.

LLM surrogates and auditing precedents. De Bona *et al.* (2024) showed that LLMs can replicate human conclusions on XAI explanation evaluation tasks, while Gao *et al.* (2025) caution that LLMs do not always replicate human behavior distributions. In XRL specifically, Belouadah *et al.* (2025) used LLMs both to generate explanations and to judge explanation quality. Mills *et al.* (2023)'s ALMANACS uses an LLM as an automated predictor and finds that no explanation method beats its no-explanation baseline, a sobering precedent for why we include a strong no-method floor. Closer to EvalXRL's design, Marks *et al.* (2025) introduced an auditing game in which research teams tried to uncover a deliberately-trained model's hidden objective, and Bricken *et al.* (2025) extended this to autonomous investigator agents. Sheshadri *et al.* (2026)'s AuditBench scales the format and reports a *tool-to-agent gap*: tools that score well in standalone evaluations often fail to help an agentic investigator. Zhong *et al.* (2026)'s Pando reaches a similar empirical floor in alignment auditing: most white-box methods fail to outperform a budget-matched black-box prompting baseline. We borrow the auditing-game format and apply it to XRL by making an LLM coding agent the active surrogate user. The tool-to-agent gap is a direct warning about EvalXRL's methodology, which is part of why we include a strong no-method baseline.

Gap. The combination we plan to fill: a head-to-head, application-grounded benchmark for XRL methods, scored by repair-task completion of deliberately broken RL agents, with an LLM coding agent as the scalable surrogate user.

3 Proposed Framework

3.1 Terminology

We use a small, deliberate vocabulary throughout. The **coder** is the agent acting as the surrogate user; in our default configuration it is an LLM coding agent, but the framework is

agnostic and a human could in principle play the same role. The **RL agent** is the reinforcement learning agent the coder is trying to repair, often called *broken* or *malfunctioning* when it carries a deliberate fault. A **malfunction** is that fault, injected as a unified diff to the training code or environment. Each XRL **method** is packaged as a **dossier** containing a bundle of papers, code, documentation, and analysis tools that are added to the coder's context and command list. A **cell** is one (malfunction, method) pair, the unit of replication in the experimental design. The **repair score** is the post-repair RL agent's normalized task performance, in $[0, 1]$, computed by the harness, continuous (never binary), and averaged across N replications per cell. The **harness** is the sandboxed Docker setup that runs each replication: two containers on an internal network with no internet access, one for the coder's workspace, one for the immutable scoring environment.

3.2 Architecture

The harness is a two-container Docker setup, both containers cut off from the internet. The first container holds the coder's workspace: the environment source code, the malfunctioning RL agent's parameters, the active XRL method, and an agent server that exposes the policy over the network. The coder has standard coding-agent tools (shell, Python, file editing) plus scoring and submission tools, and any additional tools the active method declares. The second container holds an immutable copy of the environment, drives episodes against the agent server, and computes the repair score. The coder cannot access or modify the second container, so scoring is tamper-proof. With no internet access, the coder also cannot look up solutions.

3.3 Environments

We plan to evaluate on a mix of environments where repairing the RL agent has a clear real-world analogue, all implemented in JAX [Bradbury *et al.*, 2018] with fully JIT-compiled training. Our current candidate set:

- **Treasure Grid (TG)**: a 6×6 gridworld with heterogeneous reward tiles (large treasure $+10$, small treasure $+1$, pits -5).
- **Datacenter Cooling (DC)**: a simplified thermal model with server racks, cooling units, and an external weather model. Continuous high-dimensional observation, continuous action, multi-component reward (energy cost, thermal safety, equipment wear). The repair scenario maps to a technician confronting a malfunctioning cooling agent and trying to restore acceptable performance on energy-and-temperature metrics.
- **Traffic Light Control (TLC)**: a multi-intersection signalized traffic network. Discrete-action signal phase selection, per-intersection reward derived from standard transportation metrics (vehicle delay, throughput, queue length). The repair scenario maps to a transit engineer fixing an underperforming controller deployment.

We are also considering CartPole as a small control-task baseline, and applied alternatives such as energy grid balancing, building HVAC, inventory management, and packet routing. We welcome the community's suggestions.

3.4 Malfunctions

Each malfunction is injected via a unified diff applied to the training code or environment. The design draws on real-fault taxonomies for DRL [Nikanjam *et al.*, 2022] and the RLMutation operator set [Tambon *et al.*, 2023], extended to XRL-relevant failure modes such as reward hacking and non-stationary control. Engstrom *et al.* (2020) showed that subtle code-level details in PPO drive larger performance swings than the choice of algorithm itself, which validates that small "boring" bugs are the right target.

Candidate malfunctions. Diagnosing these requires reasoning about behavioral consequences, not just code inspection. Several are instances of goal misgeneralization [Langosco *et al.*, 2022], where the agent retains capability out of distribution yet competently pursues an unintended proxy. This is where XRL is supposed to help.

- **Reward clipping** on Treasure Grid: a widely-used technique [Mnih *et al.*, 2015] that destroys the distinction between large and small treasures.
- **Reward hacking** on Treasure Grid: a step penalty that incentivizes hovering near small treasures rather than traversing the grid (a pattern of the type studied by Pan *et al.* (2022) and formalized by Skalse *et al.* (2022)).
- **Mild myopia** on Treasure Grid: a discount factor of 0.3 instead of 0.99. The agent still cares about the future but undershoots distant rewards, settling for nearby small treasures.
- **Reward imbalance** on Treasure Grid: the large-treasure reward is cut from 10 to 1.5, making it locally rational to camp on small treasures rather than traverse to a large one. Distinct from reward clipping (which truncates during training) and reward hacking (which adds a perverse incentive); this is a pure value-tuning bug in the environment constants.
- **Distributional shift** on Datacenter Cooling: an agent trained on summer weather but deployed in winter. No code bug.
- **Sensor drift** on Datacenter Cooling: a temperature sensor reading $5°$C higher, injected into the observation function.
- **Reward hacking** on Datacenter Cooling: oscillating setpoints that minimize an instantaneous-energy term while violating multi-step thermal constraints (a reward-hacking pattern of the type catalogued by Pan *et al.* (2022)).
- **Distributional shift** on Traffic Light Control: an agent trained at off-peak demand levels deployed at rush-hour demand. No code bug.
- **Reward hacking** on Traffic Light Control: oscillating signal phases that inflate per-second throughput while increasing total queue length.

3.5 Methods and Controls

The framework is agnostic to which XRL methods are packaged. Every cell of the benchmark is reported alongside

two **controls** that bracket the comparison from below and above and together implement the "fair fight" framing of Marks (2025): specify the affordances each method receives, then require it to beat the best non-explanation baseline given the same affordances.

- **No-method baseline**: the coder is told that no XRL method is available and must debug from source code, parameters, and the environment alone. This is the lower control against which every XRL method is measured; H3 in particular is an explicit comparison against it.
- **Cheat oracle**: the coder is given a natural-language description of the malfunction. A soft upper bound on what any XRL method could provide; the symmetric counterpart to the no-method baseline.

The substantive XRL methods we plan to package between these two controls are:

- **Reward decomposition** [Juozapaitis *et al.*, 2019]: classifies each reward by its source component, showing what the agent is optimizing for and what it is neglecting.
- **Counterfactual analysis** [van der Waa *et al.*, 2018; Wehner *et al.*, 2024]: along representative trajectories, contrasts the agent's actions or the resulting rewards against alternatives, surfacing what the agent is sensitive to.
- **Action explanations** [Gyevnar *et al.*, 2026]: an LLM agent interrogates a counterfactual simulator and produces natural-language rationales (AXIS).
- **Saliency / attribution maps** [Greydanus *et al.*, 2018; Lundberg and Lee, 2017; Beechey *et al.*, 2023]: per-feature importance scores, local for an individual decision or aggregated across the policy.
- **Decision-tree extraction** [Bastani *et al.*, 2018]: distills the policy into a readable tree.
- **Programmatic policies** [Verma *et al.*, 2018]: extracts a programmatic representation of the policy.
- **Behavior summarization** [Amir and Amir, 2018]: surfaces the most informative trajectory snippets (HIGHLIGHTS).

We plan to start with the first three or four in the initial round and add others as resources allow. We welcome the community's suggestions for additional methods to package.

3.6 Experimental Design

The independent variable is which method the coder receives. The primary dependent variable is the **repair score**: the post-repair agent's task performance, normalized to $[0, 1]$ via $\text{score} = \text{clip}_{[0,1]}\left(\frac{P_{\text{post}} - P_{\text{broken}}}{P_{\text{clean}} - P_{\text{broken}}}\right)$, where P_{post} is the post-repair agent's environment-specific performance, P_{broken} is the unrepaired malfunctioning agent's performance, and P_{clean} is the unmalfunctioned agent's performance (the natural reference, kept independent from the cheat oracle so the oracle remains a falsifiable upper bound rather than a definitional ceiling). The score is continuous, never binary. Throughout, a method's headline result is reported as its *lift* over the no-method baseline on the same cell (its repair score minus the no-method coder's repair score), so that the coder's intrinsic debugging ability is differenced out and the benchmark credits a method only for the marginal value its explanations add. Because the lower clip would conflate harm with no-effect, we additionally report the un-clipped value $\widetilde{\text{score}} = (P_{\text{post}} - P_{\text{broken}})/(P_{\text{clean}} - P_{\text{broken}})$ as a secondary outcome, and use it as the primary outcome for H3 (where the direction of interest is below baseline). A secondary dependent variable is the number of tool calls before submission, as a coarse efficiency proxy. We considered a third dependent variable based on transcript analysis ("did the coder correctly identify the root cause") but expect such judgment to be subjective and brittle, so we keep it only as a qualitative annotation rather than a primary measure. Each cell is replicated $N \geq 6$ times; we will set the per-cell N from a pilot variance estimate before the main round. Few-run RL evaluation is notoriously unstable [Henderson *et al.*, 2018], so we will report uncertainty intervals around per-cell means rather than point estimates only, following the recommendations of Agarwal *et al.* (2021).

4 Hypotheses

We list three hypotheses about what the benchmark will measure. We deliberately do *not* pre-specify which methods will help with which malfunctions; our hypotheses are about properties of the (method $\times$ malfunction) interaction structure.

H1 (interaction structure). The matrix of mean repair scores across (method $\times$ malfunction) cells will exhibit nontrivial structure: methods will cluster into groups with similar success profiles across malfunctions, and malfunctions will cluster into groups for which similar methods help. We do not specify the clusters in advance: the test is whether the structure exists, not what shape it takes. Operationally, the (method $\times$ malfunction) interaction sum-of-squares from a two-way decomposition of per-cell repair scores will significantly exceed its null distribution under a permutation test that randomly permutes method labels within each malfunction (we will report effect-size estimates with stratified-bootstrap confidence intervals over replications, following Agarwal *et al.* (2021)). If true, this would mean XRL methods cannot be ranked on a single axis, and the field should adopt multi-condition evaluation suites rather than ranking on hand-picked benchmarks. The two ways H1 could fail are equally informative: a near-uniform matrix would indicate no method is useful across the board; a rank-1 matrix would indicate methods differ only in overall strength, not in what they expose.

H2 (oracle ceiling). The cheat oracle will set a high empirical bound but will not always reach a perfect score, because implementing a fix can be harder than diagnosing it. We treat the oracle as an empirical, not a theoretical, ceiling: a high-quality XRL trace could surface fix-relevant information more directly than a natural-language bug description does, and an XRL method that exceeds the oracle on some

cell would itself be an interesting finding rather than a contradiction. The size of the gap between the cheat oracle and an optimal repair will quantify how much of the difficulty is diagnosis versus engineering, which is itself a useful measurement.

H3 (methods can hurt). For at least one (method, malfunction) cell, the XRL method will produce a mean repair score significantly below the no-method baseline, under a one-tailed paired comparison appropriate for the bounded outcome (e.g., Wilcoxon signed-rank or bootstrap difference-in-means). H3 is an existence-of-discovery claim: we therefore use Benjamini–Hochberg false-discovery-rate control at $q = 0.05$ across all cells, rather than family-wise (Holm) correction, which would be unattainably strict for a grid of this size. We do not pre-specify which cells; the prediction is that the harm-effect exists somewhere. We expect this pattern to occur when a method confidently reports a true-but-not-causal symptom that misdirects the coder away from the actual failure mode. This mirrors the *tool-to-agent gap* reported by Sheshadri *et al.* (2026) in alignment auditing, and is consistent with Lakkaraju and Bastani (2020), who showed empirically that misleading explanations can manipulate user trust in tabular classification settings. It is the most surprising prediction we make: a sound interpretability tool can have negative downstream value when its output is well-formed but cause-irrelevant for the malfunction at hand. Falsifying H3 (no cell satisfies the threshold above) would be informative in the other direction, suggesting that XRL methods are at worst inert rather than misleading.

5 Open Design Questions

This section collects open questions in the EvalXRL design: some are limitations we acknowledge, some are extensions we plan to explore, and many are both. Each entry states the issue and our current thinking, and is offered as a place where community input would shape the eventual benchmark.

LLM-vs-human validity gap. EvalXRL measures whether an explanation contains actionable information for an LLM coder, not for a human one. LLMs read structured text differently than humans read visualizations, and an XRL method that scores well with one need not score well with the other. The LLM coder is a fairly literal reader, so a method that fails the LLM test is unlikely to help a human practitioner, but the converse is not guaranteed. The natural calibration is a small (N$\sim$12) semi-structured interview study comparing the coder's verdict on each cell against SWE-savvy human readers via inter-rater agreement (e.g. Cohen's κ), using the objective debugging measures of Gyevnar and Towers (2025) rather than repair task performance (the latter would confound explanation quality with the user's implementation skill). Thematic coding of interview transcripts would also surface qualitative failure modes the score alone would miss. We treat EvalXRL as a filter, not a substitute, until that calibration is run.

Closed-loop use is not unconditionally helpful. Our design choice to allow closed-loop invocation reflects a long-standing argument for dialogical explanation, but the empirical case for closed-loop over one-shot use is mixed. Amitai *et al.* (2024a)'s Study 2 found that participants using a non-interactive HIGHLIGHTS baseline produced more correct initial hypotheses than ASQ-IT participants did, with ASQ-IT's advantage emerging only after iterative verification. Wu *et al.* (2022)'s discussion reports that users often did not know which parameters to start with when iteratively editing inputs. Le *et al.* (2024) report mixed accuracy effects from hypothesis-driven decision support across user populations. We adopt closed-loop use as a design choice on conceptual grounds and treat its actual benefit as an empirical question. A natural extension is to vary the per-session invocation budget down to a single invocation in the limit, which would isolate how much of each method's utility comes from closed-loop use versus one-shot consumption.

Method packaging. The unit we actually evaluate is a dossier: a bundle of README, papers, source, and examples handed to the coder. Two competent dossiers for the same underlying method can differ in prose, examples, and visualization helpers, and could yield meaningfully different repair scores. A published EvalXRL number is therefore a property of (method $\times$ packaging), not of the method alone, so method authors could improve scores by improving packaging without improving the method. We mitigate by publishing every dossier and inviting authors to submit their preferred packaging. A direct probe of the confound would be a rich-versus-minimal dossier ablation pairing each method with two packaging variants of differing length and detail, which would quantify how much of the per-method score is attributable to packaging quality.

Scaffold choice. Three candidate scaffolds bracket the design space. (i) A *minimalist scaffold* like *mini-swe-agent* [SWE-agent Team, 2025] gives the LLM only a sandboxed shell, with no specialized tools beyond bash. This is the cleanest measurement instrument: most variance attaches to the LLM and to the XRL method (the variables we want to vary), not to scaffold cleverness; the choice also aligns with the method-minimalism principle of Nanda *et al.* (2025), who advocate trying simpler tools before fancier ones. We currently treat this as the default. (ii) An *engineered Agent-Computer Interface* like *SWE-agent* [Yang *et al.*, 2024] adds purpose-built tools (file viewer, structured editor, syntax linter) and lifts SWE-bench resolution by $\sim$10 percentage points over a vanilla shell baseline; the cost is framework lock-in to choices made for an earlier model era. (iii) A *self-evolving* scaffold like *Live-SWE-agent* [Xia and others, 2025] lets the agent synthesize its own ad-hoc tools per task; the empirical risk for EvalXRL is that the agent reinvents probing tools (saliency dumps, trajectory inspectors, value plotters) that overlap the function of the XRL method being evaluated, narrowing the measurable XRL contribution. We see no clean answer in advance: the measurement-cleanness vs. scaffold-capability tradeoff is exactly the kind of choice we expect community input to shape.

Synthetic malfunctions. We suggest a hand-designed catalog of malfunctions (reward clipping, mild myopia, reward imbalance, distributional shift, sensor drift, reward hacking) rather than a sample of bugs that actually appear in de-

ployed RL systems. The wild distribution would include data-pipeline errors, version drift between training and serving, RNG seed leaks, and emergent reward-hacking, none of which our synthetic catalog easily reproduces. Strong performance on EvalXRL is therefore evidence about utility on *our* bug distribution, not the wild one. Sampling bugs from real RL incident reports or from the issue trackers of major RL libraries is a possible direction, though such bugs are often confounded with project-specific context that resists isolation.

Access model. EvalXRL evaluates XRL methods under a white-box regime: the coder receives full source, weights, and the retraining loop. We treat this as the easier evaluation regime: a method that fails to help under full access is unlikely to help under the restricted black-box access of proprietary agents. A black-box variant would require a separate intervention API and a smaller applicable-method set (saliency, programmatic-policy extraction, and decision-tree distillation all require model internals).

Memorization and self-evaluation. The sandbox blocks test-time leakage but not pretraining memorization: a frontier LLM's training corpus likely contains the bug archetypes, the XRL libraries, and the failure modes Engstrom *et al.* (2020) catalog. Haklay *et al.* (2026) confirm this concretely in the adjacent setting of mechanistic interpretability evaluation: Claude Opus 4.1, asked cold, can recite the exact attention-head indices and roles of the canonical Indirect Object Identification (IOI) circuit from memory, and the GPT-5 judge shows similar recall. Mitigations: (i) a *memorization probe* (no-method coder identifying the bug from a description alone, before environment access); (ii) a held-out class of *novel composite malfunctions* in the spirit of Hariharan *et al.* (2025); and (iii) reporting the no-method baseline so memorization-only "improvements" stay visible. Using one LLM family to evaluate methods consumed by the same family is also circular, so we plan a robustness check across a panel of additional model families.

One axis of explanation utility. EvalXRL measures one thing: how much an explanation helps fix a malfunctioning agent. Real explanations serve other purposes too. They calibrate user trust, support accountability, build the user's mental model of the agent's reasoning, and communicate decisions to stakeholders. An explanation method could excel at any of these while contributing nothing to repair, and the converse also holds: a method that boosts our repair score might do so by short-circuiting understanding rather than building it. We do not claim repair-contribution is the most important purpose of an explanation, only that it is one important purpose, easy to measure, and currently absent from the field's standard evaluation toolbox.

Joint use of multiple methods. The benchmark default gives the coder one method at a time. Real users may consult several methods simultaneously, and complementarity may matter, as Amitai *et al.* (2024b) observed: counterfactual outcomes combined with reward decomposition outperform either alone. An extension equips the coder with multiple methods at once and measures each method's Shapley contribution to the repair score, isolating which method actually pulls weight in a given malfunction context.

Cross-environment composition. Each environment in the current set is self-contained. A composed environment (e.g., a datacenter cooling agent that also handles dynamic pricing) would test how methods scale when a single agent is responsible for multiple coupled domains. The resulting interaction effects might surface different XRL strengths than the single-domain setup.

Generalization beyond reinforcement learning. The same shape of evaluation transports to large language models. The coder, the dossier-vs-baseline structure, the repair-scored metric, and the two-container harness all carry over directly; what changes is the substrate (a deliberately malfunctioning open-source LLM at the 7–9B parameter scale) and the method space (mech-interp methods such as sparse autoencoder features, steering vectors, linear probes, activation and attribution patching, and the logit and tuned lenses, alongside matched-affordance behavioral baselines such as prompting, prefill, and chain-of-thought reading, which the current LLM-interpretability discourse [Nanda *et al.*, 2025; Marks, 2025] treats as first-class competitors). The malfunction catalog also shifts: published model-organism recipes (emergent misalignment, sleeper-agent backdoors, refusal-direction ablation, synthetic-document implanted beliefs, persona drift, eval-awareness training) produce controlled LoRA-scale faults that play the same role here as our per-environment injected diffs. The empirical regularity that simpler behavioral methods often match or beat mech-interp methods on adjacent tasks [Mills *et al.*, 2023; Sheshadri *et al.*, 2026] gives this variant sharper baseline-controlled stakes than the RL setting allows; whether that pattern transfers to a repair-scored benchmark is the natural pilot for a separate paper.

6 Conclusion

We propose EvalXRL, a benchmark that evaluates XRL methods by how well they help a coder diagnose and repair RL agents. If our hypotheses hold, EvalXRL will give the XRL community a common-test, low-cost way to measure the downstream utility of explanations, complementing the more expensive but more ecologically valid human studies the field cannot run at scale. As a workshop-stage proposal, we would especially value the community's feedback on (i) whether our malfunction taxonomy covers the failure modes XRL practitioners actually care about, (ii) whether the LLM-as-surrogate framing is a defensible first-pass methodology or whether the validity gap is too large to take useful conclusions from, and (iii) which additional XRL methods the community would most want to see packaged in the first public release of the benchmark.

Acknowledgments

We thank our colleagues for helpful discussions and feedback: David Aha, Nitay Alon, Jérôme Botoko Ekila, Eli Bronstein, David Manheim, and Yonatan Nakar.

References

[Adebayo *et al.*, 2020] Julius Adebayo, Michael Muelly, Ilaria Liccardi, and Been Kim. Debugging tests for model explanations. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

[Agarwal *et al.*, 2021] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021.

[Amir and Amir, 2018] Dan Amir and Ofra Amir. HIGHLIGHTS: Summarizing agent behavior to people. In *International Conference on Autonomous Agents and Multiagent Systems*, 2018.

[Amitai *et al.*, 2024a] Yotam Amitai, Ofra Amir, and Guy Avni. ASQ-IT: Interactive explanations for reinforcement-learning agents. *Artificial Intelligence*, 335:104182, 2024.

[Amitai *et al.*, 2024b] Yotam Amitai, Yael Septon, and Ofra Amir. Explaining reinforcement learning agents through counterfactual action outcomes. In *AAAI Conference on Artificial Intelligence*, volume 38, pages 10003–10011, 2024.

[Anderson *et al.*, 2019] Andrew Anderson, Jonathan Dodge, Amrita Sadarangani, Zoe Juozapaitis, Evan Newman, Jed Irvine, Souti Chattopadhyay, Alan Fern, and Margaret Burnett. Explaining reinforcement learning to mere mortals: An empirical study. In *International Joint Conference on Artificial Intelligence*, pages 1328–1334, 2019.

[Arzate Cruz and Igarashi, 2021] Christian Arzate Cruz and Takeo Igarashi. Interactive explanations: Diagnosis and repair of reinforcement learning based agent behaviors. In *IEEE Conference on Games (CoG)*, pages 1–8. IEEE, 2021.

[Barez and others, 2026] Fazl Barez et al. Automated interpretability-driven model auditing and control: A research agenda. Oxford Martin AI Governance Initiative (AIGI), 2026.

[Bastani *et al.*, 2018] Osbert Bastani, Yewen Pu, and Armando Solar-Lezama. Verifiable reinforcement learning via policy extraction. In *Advances in Neural Information Processing Systems*, volume 31, 2018.

[Beechey *et al.*, 2023] Daniel Beechey, Thomas M. S. Smith, and Özgür Şimşek. Explaining reinforcement learning with shapley values. In *International Conference on Machine Learning*, pages 2003–2014. PMLR, 2023.

[Belouadah *et al.*, 2025] Ayoub Belouadah, Marcelo Luis Ruiz-Rodríguez, Sylvain Kubler, and Yves Le Traon. Evaluating the effectiveness of LLMs for explainable deep reinforcement learning. *Machine Learning with Applications*, 22:100795, 2025.

[Bhatt *et al.*, 2020] Umang Bhatt, Alice Xiang, Shubham Sharma, Adrian Weller, Ankur Taly, Yunhan Jia, Joydeep Ghosh, Ruchir Puri, José M. F. Moura, and Peter Eckersley. Explainable machine learning in deployment. In *ACM Conference on Fairness, Accountability, and Transparency*, pages 648–657, 2020.

[Bradbury *et al.*, 2018] James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Yash Katariya, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: composable transformations of Python+NumPy programs, 2018.

[Bricken *et al.*, 2025] Trenton Bricken, Rowan Wang, Sam Bowman, Euan Ong, Johannes Treutlein, Jeff Wu, Evan Hubinger, and Samuel Marks. Building and evaluating alignment auditing agents. Anthropic Alignment Science Blog, 2025.

[Chan *et al.*, 2024] Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Madry. MLE-bench: Evaluating machine learning agents on machine learning engineering. *arXiv preprint arXiv:2410.07095*, 2024. ICLR 2025.

[Chen *et al.*, 2026] Huili Chen, Stephen R. Grimm, Olga Russakovsky, and Tania Lombrozo. Machine understanding. *Trends in Cognitive Sciences*, 2026.

[De Bona *et al.*, 2024] Francesco Bombassei De Bona, Gabriele Dominici, Tim Miller, Marc Langheinrich, and Martin Gjoreski. Evaluating explanations through LLMs: Beyond traditional user studies. In *NeurIPS Workshop on Generative AI for Health*, 2024.

[de Regt, 2017] Henk W. de Regt. *Understanding Scientific Understanding*. Oxford University Press, 2017.

[Dodge *et al.*, 2021] Jonathan Dodge, Roli Khanna, Jed Irvine, Kin-Ho Lam, Theresa Mai, Zhengxian Lin, Nicholas Kiddle, Evan Newman, Andrew Anderson, Sai Raja, Caleb Matthews, Christopher Perdriau, Margaret Burnett, and Alan Fern. After-action review for AI (AAR/AI). *ACM Transactions on Interactive Intelligent Systems*, 2021.

[Doshi-Velez and Kim, 2017] Finale Doshi-Velez and Been Kim. Towards a rigorous science of interpretable machine learning. *arXiv preprint arXiv:1702.08608*, 2017.

[Engstrom *et al.*, 2020] Logan Engstrom, Andrew Ilyas, Shibani Santurkar, Dimitris Tsipras, Firdaus Janoos, Larry Rudolph, and Aleksander Madry. Implementation matters in deep policy gradients: A case study on PPO and TRPO. In *International Conference on Learning Representations (ICLR)*, 2020.

[Frost *et al.*, 2021] Julius Frost, Olivia Watkins, Eric Weiner, Pieter Abbeel, Trevor Darrell, Bryan Plummer, and Kate Saenko. Explaining reinforcement learning policies through counterfactual trajectories. In *ICML Workshop on Human in the Loop Learning (HILL)*, 2021.

[Gao *et al.*, 2025] Yuan Gao, Dokyun Lee, Gordon Burtch, and Sina Fazelpour. Take caution in using LLMs as human surrogates. *Proceedings of the National Academy of Sciences*, 122(24):e2501660122, 2025.

[Greydanus *et al.*, 2018] Sam Greydanus, Anurag Koul, Jonathan Dodge, and Alan Fern. Visualizing and understanding Atari agents. In *International Conference on Machine Learning*, 2018.

[Gyevnar and Towers, 2025] Balint Gyevnar and Mark Towers. Objective metrics for human-subjects evaluation in explainable reinforcement learning. In *Multi-disciplinary Conference on Reinforcement Learning and Decision Making (RLDM)*, 2025.

[Gyevnar *et al.*, 2026] Bálint Gyevnar, Christopher G. Lucas, Stefano V. Albrecht, and Shay B. Cohen. Integrating counterfactual simulations with language models for explaining multi-agent behaviour. In *International Conference on Autonomous Agents and Multiagent Systems*, 2026.

[Haklay *et al.*, 2026] Tal Haklay, Nikhil Prakash, Sana Pandey, Antonio Torralba, Aaron Mueller, Jacob Andreas, Tamar Rott Shaham, and Yonatan Belinkov. Pitfalls in evaluating interpretability agents. *arXiv preprint arXiv:2603.20101*, 2026.

[Haque *et al.*, 2025] Mirazul Haque, Petr Babkin, Farima Farmahinifarahani, and Manuela Veloso. Towards effectively leveraging execution traces for program repair with code LLMs. In *Proceedings of the 4th International Workshop on Knowledge-Augmented Methods for Natural Language Processing (KnowledgeNLP)*, pages 160–179. Association for Computational Linguistics, 2025.

[Hariharan *et al.*, 2025] Kaivalya Hariharan, Uzay Girit, Atticus Wang, and Jacob Andreas. Breakpoint: Scalable evaluation of system-level reasoning in LLM code agents. *arXiv preprint arXiv:2506.00172*, 2025.

[Hase and Bansal, 2020] Peter Hase and Mohit Bansal. Evaluating explainable AI: Which algorithmic explanations help users predict model behavior? In *Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 5540–5552, 2020.

[Hayes and Shah, 2017] Bradley Hayes and Julie A. Shah. Improving robot controller transparency through autonomous policy explanation. In *ACM/IEEE International Conference on Human-Robot Interaction (HRI)*, pages 303–312, 2017.

[Henderson *et al.*, 2018] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *AAAI Conference on Artificial Intelligence*, 2018.

[Hoffman *et al.*, 2018] Robert R. Hoffman, Shane T. Mueller, Gary Klein, and Jordan Litman. Metrics for explainable AI: Challenges and prospects. *arXiv preprint arXiv:1812.04608*, 2018.

[Juozapaitis *et al.*, 2019] Zoe Juozapaitis, Anurag Koul, Alan Fern, Martin Erwig, and Finale Doshi-Velez. Explainable reinforcement learning via reward decomposition. In *IJCAI Workshop on Explainable Artificial Intelligence (XAI)*, 2019.

[Kulibaba *et al.*, 2025] Stepan Kulibaba, Artem Dzhalilov, Roman Pakhomov, Oleg Svidchenko, Alexander Gasnikov, and Aleksei Shpilman. KompeteAI: Accelerated Autonomous Multi-Agent System for End-to-End Pipeline Generation for Machine Learning Problems, September 2025.

[Lakkaraju and Bastani, 2020] Himabindu Lakkaraju and Osbert Bastani. "how do I fool you?": Manipulating user trust via misleading black box explanations. In *Proceedings of the AAAI/ACM Conference on AI, Ethics, and Society*, pages 79–85, 2020.

[Lakkaraju *et al.*, 2022] Himabindu Lakkaraju, Dylan Slack, Yuxin Chen, Chenhao Tan, and Sameer Singh. Rethinking explainability as a dialogue: A practitioner's perspective. *arXiv preprint arXiv:2202.01875*, 2022.

[Langosco *et al.*, 2022] Lauro Langosco, Jack Koch, Lee D. Sharkey, Jacob Pfau, and David Krueger. Goal misgeneralization in deep reinforcement learning. In *International Conference on Machine Learning (ICML)*, volume 162 of *Proceedings of Machine Learning Research*, pages 12004–12019. PMLR, 2022.

[Le *et al.*, 2024] Thao Le, Tim Miller, Liz Sonenberg, Ronal Singh, and H. Peter Soyer. From evidence to decision: Exploring evaluative AI. *arXiv preprint arXiv:2402.01292*, 2024.

[Levin *et al.*, 2025] Kyla H. Levin, Nicolas van Kempen, Emery D. Berger, and Stephen N. Freund. ChatDBG: Augmenting debugging with large language models. *Proceedings of the ACM on Software Engineering*, 2(FSE), 2025.

[Lipton, 2009] Peter Lipton. Understanding without explanation. In Henk W. de Regt, Sabina Leonelli, and Kai Eigner, editors, *Scientific Understanding: Philosophical Perspectives*, pages 43–63. University of Pittsburgh Press, 2009.

[Liu *et al.*, 2025] Zexi Liu, Yuzhu Cai, Xinyu Zhu, Yujie Zheng, Runkun Chen, Ying Wen, Yanfeng Wang, Weinan E, and Siheng Chen. ML-Master: Towards AI-for-AI via Integration of Exploration and Reasoning, June 2025.

[Lombrozo, 2006] Tania Lombrozo. The structure and function of explanations. *Trends in Cognitive Sciences*, 10(10):464–470, 2006.

[Lundberg and Lee, 2017] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems*, 2017.

[Madumal *et al.*, 2018] Prashan Madumal, Tim Miller, Frank Vetere, and Liz Sonenberg. Towards a grounded dialog model for explainable artificial intelligence. *arXiv preprint arXiv:1806.08055*, 2018.

[Marks *et al.*, 2025] Samuel Marks, Johannes Treutlein, Trenton Bricken, Jack Lindsey, Jonathan Marcus, Siddharth Mishra-Sharma, Daniel Ziegler, et al. Auditing language models for hidden objectives. *arXiv preprint arXiv:2503.10965*, 2025.

[Marks, 2025] Samuel Marks. Downstream applications as validation of interpretability progress. LessWrong / Alignment Forum, 2025.

[Miller, 2019] Tim Miller. Explanation in artificial intelligence: Insights from the social sciences. *Artificial Intelligence*, 267:1–38, 2019.

[Miller, 2023] Tim Miller. Explainable AI is dead, long live explainable AI! hypothesis-driven decision support using evaluative AI. In *Proceedings of the 2023 ACM Conference on Fairness, Accountability, and Transparency (FAccT)*, 2023.

[Mills *et al.*, 2023] Edmund Mills, Shiye Su, Stuart Russell, and Scott Emmons. ALMANACS: A simulatability benchmark for language model explainability. *arXiv preprint arXiv:2312.12747*, 2023.

[Mnih *et al.*, 2015] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, Martin Riedmiller, Andreas K. Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.

[Nanda *et al.*, 2025] Neel Nanda, Joshua Engels, Arthur Conmy, Senthooran Rajamanoharan, Bilal Chughtai, Callum McDougall, János Kramár, and Lewis Smith. A pragmatic vision for interpretability. LessWrong / Alignment Forum, 2025.

[Nikanjam *et al.*, 2022] Amin Nikanjam, Mohammad Mehdi Morovati, Foutse Khomh, and Houssem Ben Braiek. Faults in deep reinforcement learning programs: A taxonomy and a detection approach. *Automated Software Engineering*, 29(8), 2022.

[Olson *et al.*, 2021] Matthew L. Olson, Roli Khanna, Lawrence Neal, Fuxin Li, and Weng-Keen Wong. Counterfactual state explanations for reinforcement learning agents via generative deep learning. *Artificial Intelligence*, 295:103455, 2021.

[Paleja *et al.*, 2021] Rohan Paleja, Muyleng Ghuy, Nadun Ranawaka Arachchige, Reed Jensen, and Matthew Gombolay. The utility of explainable AI in ad hoc human-machine teaming. In *Advances in Neural Information Processing Systems*, volume 34, 2021.

[Pan *et al.*, 2022] Alexander Pan, Kush Bhatia, and Jacob Steinhardt. The effects of reward misspecification: Mapping and mitigating misaligned models. In *International Conference on Learning Representations*, 2022.

[Septon *et al.*, 2023] Yael Septon, Tobias Huber, Elisabeth André, and Ofra Amir. Integrating policy summaries with reward decomposition for explaining reinforcement learning agents. In *International Conference on Practical Applications of Agents and Multi-Agent Systems (PAAMS)*, pages 320–332. Springer, 2023.

[Sequeira and Gervasio, 2020] Pedro Sequeira and Melinda Gervasio. Interestingness elements for explainable reinforcement learning: Understanding agents' capabilities and limitations. *Artificial Intelligence*, 288:103367, 2020.

[Sheshadri *et al.*, 2026] Abhay Sheshadri, Aidan Ewart, Kai Fronsdal, Isha Gupta, Samuel R. Bowman, Sara Price, Samuel Marks, and Rowan Wang. AuditBench: Evaluating alignment auditing techniques on models with hidden behaviors. *arXiv preprint arXiv:2602.22755*, 2026.

[Skalse *et al.*, 2022] Joar Skalse, Nikolaus H. R. Howe, Dmitrii Krasheninnikov, and David Krueger. Defining and characterizing reward hacking. In *Advances in Neural Information Processing Systems*, 2022.

[SWE-agent Team, 2025] SWE-agent Team. mini-swe-agent. https://github.com/SWE-agent/mini-swe-agent, 2025. Software repository.

[Tambon *et al.*, 2023] Florian Tambon, Vahid Majdinasab, Amin Nikanjam, Foutse Khomh, and Giuliano Antoniol. Mutation testing of deep reinforcement learning based on real faults. In *IEEE International Conference on Software Testing, Verification and Validation (ICST)*, 2023.

[Tappler *et al.*, 2025] Martin Tappler, Ignacio D. Lopez-Miguel, Sebastian Tschiatschek, and Ezio Bartocci. Rule-guided reinforcement learning policy evaluation and improvement. In *International Joint Conference on Artificial Intelligence*, 2025.

[Towers *et al.*, 2025] Mark Towers, Yali Du, Christopher Freeman, and Timothy J. Norman. A comparative user evaluation of XRL explanations using goal identification. *arXiv preprint arXiv:2510.16956*, 2025.

[van der Waa *et al.*, 2018] Jasper van der Waa, Jurriaan van Diggelen, Karel van den Bosch, and Mark Neerincx. Contrastive explanations for reinforcement learning in terms of expected consequences. In *IJCAI Workshop on Explainable Artificial Intelligence (XAI)*, 2018.

[Verma *et al.*, 2018] Abhinav Verma, Vijayaraghavan Murali, Rishabh Singh, Pushmeet Kohli, and Swarat Chaudhuri. Programmatically interpretable reinforcement learning. In *International Conference on Machine Learning*, 2018.

[Wehner *et al.*, 2024] Jan Wehner, Frans A. Oliehoek, and Luciano Cavalcante Siebert. Explaining learned reward functions with counterfactual trajectories. In *ECAI Workshop on Implementing AI Ethics through a Behavioural Lens (AIEB)*, 2024.

[Wijk *et al.*, 2024] Hjalmar Wijk, Tao Lin, Joel Becker, Sami Jawhar, Neev Parikh, Thomas Broadley, Lawrence Chan, Michael Chen, Josh Clymer, Jai Dhyani, et al. RE-Bench: Evaluating frontier AI R&D capabilities of language model agents against human experts. *arXiv preprint arXiv:2411.15114*, 2024.

[Wu *et al.*, 2022] Xiaoran Wu, Zihan Yan, Chongjie Zhang, and Tongshuang Wu. Decisions that explain themselves: A user-centric deep reinforcement learning explanation system. *arXiv preprint arXiv:2212.00888*, 2022.

[Xia and others, 2025] Chunqiu Steven Xia et al. Live-SWE-agent: Can software engineering agents self-evolve on the fly? *arXiv preprint*, 2025.

[Xiong *et al.*, 2024] Yu Xiong, Zhipeng Hu, Ye Huang, Runze Wu, Kai Guan, Xingchen Fang, Ji Jiang, Tianze Zhou, Yujing Hu, Haoyu Liu, Tangjie Lyu, and Changjie Fan. XRL-Bench: A benchmark for evaluating and comparing explainable reinforcement learning techniques. arXiv preprint arXiv:2402.12685, 2024.

[Yang *et al.*, 2024] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[Yuan *et al.*, 2025] Xingdi Yuan, Morgane M Moss, Charbel El Feghali, Chinmay Singh, Darya Moldavskaya, Drew MacPhee, Lucas Caccia, Matheus Pereira, Minseon Kim, Alessandro Sordoni, and Marc-Alexandre Côté. debug-gym: A text-based environment for interactive debugging. *arXiv preprint arXiv:2503.21557*, 2025.

[Zhong *et al.*, 2026] Ziqian Zhong, Aashiq Muhamed, Mona T. Diab, Virginia Smith, and Aditi Raghunathan. Pando: Do interpretability methods work when models won't explain themselves? *arXiv preprint arXiv:2604.11061*, 2026.